Pythonic Data Structures

Welcome back to CS 2100!

Prof. Rasika Bhalerao

List comprehension can be used for things that aren't lists

Iterate over a string and create a set:

```
phrase: str = 'never gonna give you up'

letters: set[str] = {letter.lower() for letter in phrase}

print(letters)  # {'v', 'g', 'i', 'o', 'n', 'a', 'y', 'p', 'u', 'e', 'r', ' '}
```

It is always up to you to decide which version is easiest to read for your code. Sometimes, a basic `for` loop is more readable.

Poll: What does this function do (other than confuse)?

```
def something(docs: list[str]) -> int:
    """Confuses students.

    Parameters:
        docs : list[str]
            A list of confusing strings, each more ridiculous than the last

    Returns:
        int
            A confusing number
    """
    return sum(len(open(doc, encoding="utf-8").read()) for doc in docs)
```

Tuple: like a list, but immutable (i.e., its contents cannot be changed after declaration)

What is the `type` of the collection of arguments `*args`? A `tuple`!

```
def print_args(*args: T) -> None:
    """Print each argument on a separate line"""
    for item in args:
        print(item)

print_args(1, 2, 3)
```

```
nums: tuple[int, int, int] = (1, 2, 3)

my_long_tuple: tuple[int, ...] = tuple([-i for i in range(50)])
```

Tuple: like a list, but immutable (i.e., its contents cannot be changed after declaration)

We can sort a list in-place, but not a tuple.

```
my_list.sort()
print(my_list) # [-400, 1, 2, 3]

my_tuple.sort() # impossible
```

Poll: Tuples and lists are very similar, but we can't modify tuples. Which of these collections should be a tuple instead of a list?

1. Students registered for a course
2. Cats in a shelter
3. The seven days of the week
4. Driving directions from school to the airport (turn left, drive 2 miles, ...)

The tricky part:

- The tuple is immutable
- The variable `my_tuple` (the pointer to the location in the computer's memory) is a mutable variable

We can't mutate the tuple, but we can re-assign the variable `my_tuple` to a sorted version of the same tuple:

```
my_long_tuple: tuple[int, ...] = tuple([-i for i in range(5)])
my_long_tuple = tuple(sorted(my_long_tuple))
print(my_long_tuple)      # (-4, -3, -2, -1, 0)
```

Poll: Which ONE is not allowed? (Hint: `str`s are immutable)

```
my_str: str = 'mini'
```

1. `print(my_str.upper())`
2. `my_str = my_str.upper()`
3. `my_str = 'MINI'`
4. `my_str[0] = 'B'`

Sets

A set is very similar to a list: it is a collection of items.

```
words: set[str] = {'hi', 'hi', 'hello', 'hi', 'howdy', 'hi'}

print(words)  # {'hi', 'hello', 'howdy'}
```

Differences between a set and a list:

- A set is unordered
- A set can only hold each item (at most) once -- no duplicates

Exercise

Let's write a function that takes a `str` and counts the number of unique (distinct) words in it.

Solution

```
def count_unique_words(text: str) -> int:
    return len(set(text.split()))

print(count_unique_words('hello hi hi hello howdy hi'))  # 3
```

Some set syntax

Creating a set:

```
words: set[str] = {'hi', 'hi', 'hello', 'hi', 'howdy', 'hi'}

numbers: set[int] = set(range(5))
print(numbers)  # {0, 1, 2, 3, 4}

list_of_floats: list[float] = [3.4, 3.2, 2.9, 3.4, 3.0]
measurements: set[float] = set(list_of_floats)
print(measurements)  # {3.2, 3.0, 2.9, 3.4}
```

Some set syntax

Adding and removing items, iterating over a set, and getting its size:

```
nums: set[float] = set()

for i in range(100):
    random_float = round(random(), 2) # random float rounded to nearest hundredth
    nums.add(random_float)

print(len(nums))

numbers: set[int] = set(range(5))
numbers.remove(3)
print(numbers)  # {0, 1, 2, 4}
```

Exercise

Let's write a function that checks if any two people in this room have the same birthday. It should have a loop that iterates (up to) 40 times.

Each iteration, it should:

- Ask the user to input their birthday via two separate `int`s: the month and the day (ask twice to get the two `int`s)
- Store their birthday as a tuple
- If that birthday is already in the set, return `True`
- If not, add it to the set

After the loop (which it should only reach if no two people have the same birthday), it should return `False`.

Solution

```
num_students: int = 80

def any_same_birthdays() -> bool:
    birthdays: set[tuple[int, int]] = set()

    for _ in range(num_students):
        month: int = int(input('Please enter the month as a number between 1 and 12: '))
        day: int = int(input('Please enter the day as a number between 1 and 31: '))
        date: tuple[int, int] = (month, day)
        if date in birthdays:
            return True
        else:
            birthdays.add(date)

    return False
```

Some set syntax

Binary set operations:

- Union (`a | b`): a set that has all elements that are in either set `a` or set `b`
- Intersection (`a & b`): a set that has all elements that are in both set `a` and set `b`
- Subset (`a <= b`): `True` if all elements in `a` are also in `b`, and `False` otherwise
  - Strict subset (`a < b`): `True` if `a <= b` **and `a` is not equal to `b`**, and `False` otherwise
- Subtraction (`a - b`): a set that has all elements in `a` that are not in `b`

```
nums_a: set[int] = set(range(1, 5))
nums_b: set[int] = set(range(3, 9))

print(nums_a | nums_b)  # {1, 2, 3, 4, 5, 6, 7, 8}
print(nums_a & nums_b)  # {3, 4}
print(nums_a <= nums_b) # False
```

Poll: Why is there no binary "Addition" operation for sets? (There is Subtraction.)

1. Because it would be the same as the Intersection operation
2. Because it would be the same as the Union operation
3. Because it would be the same as the Subtraction operation
4. There is an Addition operation

Dictionaries

We use curly brackets (`{` and `}`) to represent sets. But we also use them to represent dictionaries:

```
print(type({'hello'}))  # <class 'set'>
print(type({}))         # <class 'dict'>
```

Curly brackets, when empty (or non-empty, but formatted a specific way), denote a dictionary.

A dictionary is also known as an "associative array".
It's like a list, but the indices are not required to be contiguous ints -- the indices can be of any type

A dictionary maps key --> value

Each key can appear at most once (the keys are a set)

Here are two examples which map each animal (`str`) to their age (`int`):

```
ages: dict[str, int] = {'elephant': 12, 'cat': 10}
print(ages)  # {'elephant': 12, 'cat': 10}

also_ages: dict[str, int] = dict([('elephant', 12), ('cat', 10)])
print(also_ages)  # {'elephant': 12, 'cat': 10}  (same as before)
```

Exercise

Let's write a function that takes a `str` and returns a dictionary that maps from each unique word in the `str` to the number of times it appears.

Solution

```
def word_counter(text: str) -> dict[str, int]:
    word_counts: dict[str, int] = dict()
    for word in text.split():
        word_counts[word] = word_counts.get(word, 0) + 1
    return word_counts

print(word_counter('hello hi hi hello howdy hi'))  # {'hello': 2, 'hi': 3, 'howdy': 1}
```

Some dictionary syntax

Access a value given a key:

- brackets (`[key]`)
- `get(key)` method
    - handles the case if the `key` is not in the `dict`

```
ages: dict[str, int] = {'elephant': 12, 'cat': 10}

print(ages['cat'])  # 10
print(ages.get('cat'))  # 10
print(ages.get('dog'))  # None
print(ages.get('dog'), 3)  # 3
print(ages['dog'])  # raises KeyError
```

Some dictionary syntax

If we add the same `key` twice, it overwrites the original `value` with the second `value`.

```
ages: dict[str, int] = {'cat': 10}

ages['elephant'] = 12
print(ages)  # {'cat': 10, 'elephant': 12}

ages.update([('elephant', 13)])
print(ages)  # {'cat': 10, 'elephant': 13}

ages['elephant'] = 14
print(ages)  # {'cat': 10, 'elephant': 14}

ages.update([('dog', 3)])
print(ages)  # {'cat': 10, 'elephant': 14, 'dog': 3}
```

Exercise

Let's write a function that helps us with Scrabble.

- A very common situation: We are playing Scrabble. We see we have 3 'O's. What can we do?
- The plan: get a map that gives us options based on a letter
- Let's write a function that takes a letter as a parameter and returns a dictionary where:
  - The keys are all possible frequencies of that letter (except zero)
  - The values are the sets of words in the dictionary with that many of that letter
- Here's a list of english words if you need one (the official Scrabble list is harder to get as a text file)

Solution

```
def scrabble_helper(letter: str) -> dict[int, set[str]]:
    result: dict[int, set[str]] = dict()
    with open('/path/to/dictionary.txt', 'r', encoding='utf-8') as english_dict:
        for word in english_dict.readlines():
            if letter in word:
                word = word.strip()
                letter_count = word.count(letter)
                if letter_count in result:
                    result[letter_count].add(word)
                else:
                    result[letter_count] = {word}
    return result

result: dict[int, set[str]] = scrabble_helper('r')

for key, value in result.items():
    if key > 2:
        print(f'{key}: {value}')
```

Some dictionary syntax

Iterate over a `dict`:

```
ages: dict[str, int] = {'cat': 10, 'elephant': 14, 'dog': 3}
```

- over its `key`s

```
for key in ages:
    print(f"{key}'s age is {ages.get(key)}")
```

- over its `key-value` pairs

```
for key, value in ages.items():
    print(f"{key}'s age is {value}")
```

JSON (JavaScript Object Notation)

- Popular format for storing data
- Very common for APIs to send us data in JSON format
  - E.g., https://openweathermap.org/api/one-call-3
- Read as a dictionary

```
import json, pprint

with open('example_json_data.json', 'r', encoding='utf-8') as f:
    data = json.load(f)
    pprint.pp(data)
```

Notes to run the previous example on your own later

We took the example API response from the Weather API and stored it in a file called `example_json_data.json`.

- Removed the lines with ellipses (`...`)
- Removed the commas on the lines before them
- Added an ending bracket (`}`).

`pprint` (https://docs.python.org/3/library/pprint.html) is a library for printing data in a readable format.

Poll: Which data structure is best suited for this task?

Creating a product that works differently on different operating systems, and we want to know which operating systems we need to support

1. List
2. Tuple
3. Set
4. Dictionary

Poll: Which data structure is best suited for this task?

Storing the order in which young children should stand in line

1. List
2. Tuple
3. Set
4. Dictionary

Poll: Which data structure is best suited for this task?

Storing the 7 days of the week (Sunday, Monday, Tuesday, ..., Saturday)

1. List
2. Tuple
3. Set
4. Dictionary

Poll: Which data structure is best suited for this task?

Keeping track of each student's favorite color

1. List
2. Tuple
3. Set
4. Dictionary

Poll:

1. What is your main takeaway from today?

2. What would you like to revisit next time?